SUMMARY: ADVANCED PROMPT ENGINEERING TECHNIQUES

Objective:

By the end of this session, students will be able to:

- Understand the limitations of basic prompting techniques and why advanced strategies are necessary for complex reasoning tasks.
- Learn and apply advanced prompting methods such as Chain of Thought (CoT), Self-Consistency, and Tree of Thought (ToT).
- Practice using Zero-Shot, Few-Shot, and Auto-CoT approaches to guide the model's step-by-step reasoning process.
- Recognize when to use CoT, Self-Consistent CoT, or Tree of Thought based on the nature of the problem—whether it requires logical deduction, stable outputs, or creative exploration.
- Compare linear and branching reasoning strategies and understand how structured exploration improves output quality in open-ended or multi-solution tasks.
- Develop an intuition for matching task complexity with the appropriate prompting strategy to improve model reliability, clarity, and creativity.

We've spent the last session exploring the **core prompting techniques** —
Zero-Shot, One-Shot, Few-Shot, and Role-Based prompting.
These gave us the tools to **guide the AI**, teach it with **examples**, and shape its **tone and behavior**.

They're like giving instructions to a **new teammate**:
You either tell them what to do directly, give them examples, or ask them to think like a certain expert.

But here's the thing...

Imagine you're working with an intern:

- At first, you tell them: "Just do this."
- Then you show them a couple of examples.
- Eventually, you ask them to think like a content writer or a recruiter or anything else..

That's basic prompting — task-level instructions.

But what happens when:

- The task gets complex?
- They need to **analyze**, **plan**, or **make decisions**?
- You want them to **explain how they reached the answer**, not just give one?

That's when **basic prompting falls short** — and you need to train them to **think**, not just act.

This is Where Advanced Prompting Techniques Come In.

Now, we're moving into techniques that make the AI behave more like a **thinking assistant**, not just a responsive one.

You'll learn how to:

- Let it **decide when to think and when to act** (ReAct)
- Make the model **reason step-by-step** (Chain-of-Thought)
- Improve accuracy by comparing multiple answers (Self-Consistency)
- And explore **multiple paths to a solution** (Tree-of-Thought)

Limitations of Basic Prompting

Let's say the task becomes more than "just do this."
You now want the model to:

- **Analyze** before answering
- **Plan** a course of action
- **Explain** their decision-making
- **Handle uncertainty** or incomplete information
- **Avoid shortcut errors**

But here's what happens when we rely only on basic prompting:

1. No Reasoning Path → Blind Guessing

Example:
Prompt: "A man has 4 sons. Each son has a sister. How many children does he have?"

Basic Prompt → "20"
(Mistakenly assumes 4 sons × 1 sister each = 4 daughters → total = 8 kids)

Correct Answer: 5
(They all share one sister)

Without step-by-step reasoning, the model **guesses**, not **thinks**.

2. Fails on Multi-Hop Logic

Prompt: "If it's raining and John forgot his umbrella, will he get wet?"

Basic Prompt → Might say: "No" or "Unclear"

Why?
It doesn't connect **raining → no umbrella → exposed to rain → gets wet**

3. Not Transparent (Black Box)

When a model gives just an answer like "42" — you don't know *how* it got there.

This is risky for:

- Healthcare
- Legal reasoning
- Education
- Decision-making systems

Humans trust systems that **show their work**, not just the result.

4. Doesn't Generalize Well

Basic prompting relies on surface-level patterns.

Change the phrasing slightly, and the model might fail:

- "Add 3 apples and 2 bananas" → OK
- "Add 2 fruits and 3 more fruits" → Might stumble

Why?
Because it hasn't been *taught to reason*, only to *match patterns*.

5. Breaks Down in Ambiguous or Open-Ended Scenarios

Example:

> "Write a strategy to reduce pollution in a smart city using AI."

With basic prompting, the model might give a **shallow, generic** answer.
It doesn't plan, evaluate alternatives, or show structured thought.

Now, What's the Solution?

When your intern (or AI assistant) faces a tough problem —
You don't just want them to "act" — you want them to "**think**."

That's where **Advanced Prompting Techniques** come in.
These techniques teach the model how to think, plan, reflect, and justify — just like a good teammate or student would.

Chain of Thought(COT):

Before understanding Chain of Thought(COT), we need to understand why we need COT, what things simple prompting does not solve.

Imagine you're working with two interns.
You ask both:

> "How many pens will we need for a 3-day workshop if each participant needs 2 pens per day and we're expecting 25 participants?"

- **Intern A** replies: *"150"*
- **Intern B** replies:
  "25 people × 2 pens = 50 pens per day.
  50 × 3 days = 150 pens."

Who do you trust more? Obviously, **Intern B**, because they **showed their thinking**.

You didn't just get the answer — you got the **reasoning behind the answer.**

That's exactly what we want from LLMs — not just output, but **explainable reasoning**.

Chain of Thought (CoT) prompting is a technique where we **encourage the model to think step-by-step** before giving a final answer, just like humans do when solving complex problems.

Rather than jumping directly to a response, the model is asked to:

- Break down the question
- Process each part logically
- Reach a conclusion based on those steps

Chain of thoughts (COT)

What Happens Inside the Model When You Use CoT Prompting?

Let‘s go one level deeper and understand what happens inside the model when we type:
“Let’s think step by step.”

Imagine you’re an actor trained on millions of movie scripts.

- If the director says:
  “Play it serious.” → You use a deep tone, speak slowly, and keep a straight face.
- If they say:
  “Play it silly.” → You use exaggerated gestures and a funny voice.

Similarly, LLMs **don’t understand**, but they’re *really good at mimicking patterns*.
And during training, they’ve seen **tons of examples** where “Let’s think step by step…” is followed by **logical, multi-step reasoning.**

So when you use that phrase—or any “reasoning cue” — it triggers the model to **follow a different pattern**.

Step 1: LLMs don’t "think" — they predict.

Large Language Models (LLMs) like ChatGPT aren't thinking in the human sense.
They are **predicting the next word** in a sequence — word by word — based on patterns they've seen during training.

Step 2: Prompts Act Like "Lenses" or "Modes of Thinking"

When you change the *instruction*, you change the *lens* through which the model views the task.

- Prompt: *"What's 17 × 12?"*
  Model sees this as:
  "Just answer. Be short."

- Prompt: *"Let's think step by step."*
  Model sees this as:
  "I've seen this phrase before → It usually leads to multi-line reasoning → I'll start writing an explanation before giving the answer."

  The magic? That *one phrase* nudges the model into a different "reasoning mode."

But CoT isn't just a single trick.
It's more like a toolkit — with different tools for different kinds of reasoning tasks.

Just as in our real life, we don't always use the same thinking process.
Sometimes we figure things out without examples.
Sometimes we learn by seeing examples.
Other times, we try different approaches and compare.
That's how *humans* think — and that's how CoT works as well.

Now, let's go one layer deeper and explore the **Types of Chain of Thought** — each with its own use case, structure, and purpose."

Types of COT:

So let's begin with the first — and most common — **Zero-shot Chain of Thought**…

1. Zero Shot Chain of Thought:

In machine learning, **"zero-shot"** means:

> "Solving a task **without seeing any examples** of that task during the prompt."

So here, we're asking the model to *reason*, without first teaching it *how* to reason in that specific situation.

Imagine giving a new intern a task and just saying:

"Think through the steps carefully before you answer."

You didn't show them examples.
You didn't walk them through it.
You just encouraged them to use logic.

That's **Zero-shot CoT.**

Zero-shot CoT is the **simplest** and most widely used form of Chain of Thought prompting.

In this method, you **don't provide any examples or demonstrations** beforehand.
Instead, you just give the model a reasoning cue — most commonly:

"Let's think step by step."

This small phrase **nudges the model** to enter a reasoning mode based on its training.
It has learned that this cue is **usually followed by logical, step-by-step responses** — so it imitates that structure.

For example:
Prompt: If a train travels 300 miles in 5 hours, what is its average speed? Let's think step by step.
Model Response:

Step1: Identify Given Values:
Distance = 300 miles
Time = 5 hours

Step 2: Formula:
Speed = Distance / Time

Step 3: Calculation:
Speed = 300 miles ÷ 5 hours = 60 mph

Final Answer:
The train's average speed is 60 mph.

Zero-shot CoT gives us a massive improvement over basic prompting —
It encourages the model to reason.
But just like a student thinking for themselves, it **still has gaps**."

Limitations of Zero-shot CoT

1. No Guidance on Format or Style

Even if you say *"Let's think step by step,"* the model decides **how** to reason.
Sometimes it breaks it down nicely… sometimes it doesn't.

Inconsistency in reasoning flow
Different answers for same question across attempts

2. Fails on Unfamiliar or Uncommon Tasks

Zero-shot CoT only works if the model has **seen enough examples of similar reasoning during training.**

If the task is rare, creative, or outside training data — it might still hallucinate.

3. No Control Over Quality of Reasoning

You can't ensure:

- Are the steps actually logical?
- Is the thinking clear and structured?
- Did the model **copy a wrong pattern**?

It might still shortcut or skip steps.

4. Not Adapted to Specific Contexts

Zero-shot CoT gives general reasoning behavior.
But what if you want a very **specific** way of solving a problem?

For example:

- A specific math strategy?
- A structured template for business logic?

Zero-shot CoT **can't follow your custom pattern** unless you show it.

So, what do we do when we want to improve all that?
Just like teaching a student — instead of hoping they'll figure it out, we show them examples of how to think."

"*That's exactly what Few-shot Chain of Thought does.*
It teaches the model by demonstration, not just instruction.

2. Few-shot chain of prompting:

In AI, **"few-shot"** means:

Solving a new task by showing **a few solved examples** of similar tasks first.

Unlike zero-shot CoT (which only gives the cue), here we **guide the model more explicitly** by showing it exactly how we want it to think.

Imagine teaching a new intern how to write a report:

- First, you give them a few sample reports (with reasoning).
- Then, you ask them to write a new one.

Now they're **not guessing**, they're **replicating a thinking pattern** — that's Few-shot CoT.

Few-shot CoT involves giving the model **one or more examples** of how to reason step-by-step **before** asking it to solve a new problem.

This helps the model learn the **structure of reasoning** by seeing examples, just like a student would learn by watching a teacher solve a few questions on the board.

For Example:

You are an intelligent HR assistant. Read the candidate's background and recommend the best-fit job title based on their skills and experience.

Use the following format:

Resume: <resume text>
Reasoning: <explain your reasoning>
Job Title: <final prediction>

Example 1:

Resume: Experienced in Python, NumPy, and data visualization using Matplotlib. Built dashboards and conducted exploratory data analysis in financial sector.
Reasoning: The candidate has experience in data analysis, Python, and visualization tools commonly used in analytics roles.
Job Title: Data Analyst

Example 2:

Resume: Worked on React, Node.js, and REST APIs. Led frontend redesign and integrated backend services for a SaaS product. Focus on user experience and performance.
Reasoning: The candidate specializes in frontend and full-stack development using modern web technologies.
Job Title: Frontend Engineer

—

Resume: Worked on natural language processing tasks using Hugging Face Transformers. Built question-answering pipelines, used spaCy for entity recognition.

Reasoning:

Few-shot CoT is powerful. It lets us guide the model with curated examples and show it exactly how we want it to reason.

But — as with any method — it comes with its own challenges."

Limitations of Few-shot CoT

1. Manual Effort – You Have to Write the Examples Yourself

- Every time you create a new prompt, you need to manually:
  - Pick the right example problems
  - Write high-quality reasoning steps
  - Format everything consistently

It works.But it doesn't scale.

2. Finding Good Examples Can Be Hard

- What if you're working in a **new domain**?
- What if you're not sure how to structure the reasoning yourself?
- What if the task has **no clear "correct" format**?

The effectiveness of Few-shot CoT depends **entirely on the quality of your examples**.

3. Examples Can Bias the Model Too Much

- If your examples are too specific or rigid,
  - The model might just copy them blindly, even when it's not appropriate.
  - Or apply the wrong structure to a different kind of problem.

It's like showing a student how to solve one kind of puzzle —
And then they apply that *same exact method* to every other question, even when it doesn't fit.

Let's understand this with an example:

Prompt 1 (With FEW SHOT COT) -

"We are analyzing the following case and want to predict the fraud type based on given descriptions. Here's how we reason through the examples:

Example 1:

Case Description: A customer makes a series of small withdrawals, each under the reporting threshold, but the total amount is significant.

Fraud Options: Money laundering, Loan stacking, Identity theft.

Reasoning: Since the customer is attempting to avoid detection by keeping each withdrawal small, the most likely fraud type is money laundering.

Fraud Type: Money laundering.

Example 2:

Case Description: A person uses multiple accounts to transfer funds between them before withdrawing them in cash.

Fraud Options: Money laundering, Loan stacking, Identity theft.

Reasoning: The individual is exploiting a system designed to track large transactions. The most likely fraud type is money laundering.

Fraud Type: Money laundering.

Example 3:

Case Description: A customer takes out multiple loans from different financial institutions and fails to repay them.

Fraud Options: Money laundering, Loan stacking, Identity theft.

Reasoning: This is an example of loan stacking, where the individual takes out loans from multiple institutions without the intent to repay.

Fraud Type: Loan stacking.

Now, consider the following case:

New Case Description: A customer deposits large sums of money into their account and immediately withdraws them in smaller amounts.

Fraud Options: Money laundering, Loan stacking, Identity theft.

Reasoning: Based on the examples provided, this pattern seems to involve small withdrawals after a large deposit, which resembles money laundering because it tries to avoid detection by breaking the large amount into smaller parts.

Fraud Type: (Please output the most likely fraud type.)"

But now we are using a different prompt without Few Shot COT:

PROMPT 2 (WITHOUT A FEW SHOTS):

"We are analyzing the following case and want to predict the fraud type based on the given descriptions. Please reason through the case logically and select the most appropriate fraud type.":

Case Description: A customer deposits large sums of money into their account and immediately withdraws them in smaller amounts.

Fraud Options: Money laundering, Loan stacking, Identity theft, Structuring, and Fraudulent transfer.

Reasoning: Think through the fraud types that could be associated with the description provided. What type of fraud fits best based on the behavior described in the case?

Fraud Type: (Please output the most likely fraud type.)"

You will see a different output in the case of a prompt without Few Shot COT and a prompt with Few Shot COT

They got *biased* by our examples. Instead of learning the **general approach**, they copied the structure blindly.

This is exactly what happens with LLMs when we give them very specific examples in a prompt. They might:

- Repeat the style or format rigidly,
- Apply the same method even when it's not appropriate,
- Miss the underlying instruction or variation.

So while examples are useful, **too-specific examples can mislead the model.**

4. Not Flexible for Large-Scale or Automated Use

- What if you have **hundreds of tasks**?
- Or dynamic data where prompts need to be generated automatically?

Few-shot CoT doesn't scale in these contexts, because you can't keep hand-writing examples for everything.

Let's understand this by a scenario where you are generating the product which will help in categorizing the invoice generated, and you are building this application using few few-shot COT

Scenario: Automating Invoice Categorization

Problem:

A financial software company wants to categorize thousands of invoices into expense types like Travel, Meals, Office Supplies, Software, etc., using an LLM.

Prompt (Few-shot CoT):

You are an expert financial assistant. Categorize the invoice based on description and give a reason.

Examples:

1. Description: "Uber ride from client meeting"
   Category: Travel
   Reason: Uber is used for transportation to a meeting.

2. Description: "Lunch with marketing team at Café Rio"
   Category: Meals
   Reason: Food expense incurred during team lunch.

Now, categorize this:

Description: "Adobe Creative Cloud subscription renewal"

Expected Output:

Category: Software

Reason: Adobe Creative Cloud is a software tool used for creative work.

Now, discuss the **limitations of building this application:**

1. This prompt works well for a **small, fixed set of examples**.

2. But when deployed across **10,000+ invoices**, where each line item varies ("Train ticket to Bangalore", "Cloudflare annual billing", "Printer paper purchase", etc.), it becomes **infeasible to manually include representative CoT examples** for **every variation**.

3. If new expense types emerge (e.g., "AI API usage billing"), the model **might fail** without being explicitly shown a similar example.

4. Maintaining these **hard-coded few-shot examples becomes a bottleneck**.

So, now the question is, rather than manually writing examples for every domain or style, **can the model generate its own reasoning paths automatically, and still be reliable?**

That's where **Auto-CoT** comes in

It removes the manual burden and makes the model act like its own tutor.
With Auto-CoT, the model learns how to solve a task and then generates **its own reasoning examples**, which can then be reused in other prompts.

This makes it scalable, automated, and super powerful — especially for large or unfamiliar problem domains.

3. Auto Chain of Thought:

In traditional Few-shot CoT, **you** write the examples.

In Auto-CoT, **the model itself**:

1. Understands the type of reasoning task
2. Generates a few step-by-step solutions (on request)
3. Then apply those to new queries

It becomes its own tutor.

Let's say you hire a junior analyst.

- You give them a new type of data problem they've never solved before.
- Instead of you giving them sample reports...
- You say, "Why don't you try a few practice reports first?"
- Then you pick the best ones and use them as the base for future tasks.

> That's Auto-CoT — **learning by self-generated example**

Auto-CoT stands for Automatic Chain of Thought prompting.

In this method, we ask the model to generate reasoning examples by itself, which can then be reused in few-shot COT style prompts.

> Instead of manually crafting few-shot reasoning examples, the model creates its own and then uses them to solve new problems.

How Auto-CoT Works:

Auto-CoT, or Automatic Chain of Thought, works in two major phases — one where the model generates its own reasoning examples, and the other where it applies those examples to solve new problems.

Think of this process as the model becoming both the teacher and the student.

Let's break it down step by step:

Phase 1: The Model Creates Its Own Reasoning Examples

You give a general instruction, like:
"Generate 3 example math questions with step-by-step reasoning."

You're not giving any examples yourself —
You're **asking the model to create the examples first.**

What Happens Internally?

During training, the model has seen tons of:

- Math problems
- Word problems
- Logical puzzles
  ...and their solutions written in a "step-by-step" format.

So when you ask it to **generate new examples**, it draws on those training patterns and mimics how such problems are usually solved.

Goal of This Phase:

You're letting the model **bootstrap** its own reasoning style —
essentially saying:

> "You know how problems like this are usually solved — now show me how *you* would explain them."

For Example:
Prompt:
Imagine a junior data analyst at a consulting firm who's just been assigned to analyze a client's latest customer data set.

But before touching the client's data, the analyst takes a self-initiated warm-up step:

1. They pull up similar past datasets from internal projects.
2. They practice:
   - Writing 2–3 mock summaries
   - Running some exploratory analysis
   - Drafting charts and insights based on what they've seen before.

Why do they do this?

Because it primes their thinking, sharpens their patterns, and reminds them:

- How to structure the analysis

- What kind of metrics matter
- What good insights typically look like

Only after this warm-up does the analyst move on to the real client data, now with a much clearer mental model and reasoning structure.

But wait, we start with nothing, like we just give a prompt, so what is the difference between Zero-shot COT and phase 1 of Automated COT

So at first glance, it *does* look like Zero-Shot CoT — you're just giving the model a prompt and letting it respond, right?

But here's the key difference:

> In Zero-Shot CoT, you give the model the **target question** directly and hope it solves it correctly using step-by-step reasoning.

What we just did means that in Phase 1, we're **not asking it to solve the target task yet**.
We're telling it:

> "Before we begin, generate your own similar problems — and solve them step by step."

So what's actually happening?

- The model is **bootstrapping its own reasoning context**.
- It's effectively **reminding itself how this type of problem is normally approached**.
- This **creates an internal 'few-shot' context**, but it's made by the model itself.

That's the real difference:
So you're still "prompting with nothing," yes — but you're using that one prompt to **prime the model**, not to answer right away.

It's a warm-up, not the test.

Why This Works:

The model:

- Reinforces the reasoning style it just used
- Keeps its internal structure fresh in a short-term context
- Doesn't rely on *you* for examples — it **teaches itself**

Phase 2: The Model Applies Its Own Reasoning Style

Now that the model has created 2–3 reasoning examples, you **give it a new question** and ask it to solve it in the **same way**.

You've moved from "Generate examples" to "Apply this format."

Example:

So now our Junior Analyst handles Real Client Data

After warming up with mock data from internal projects, the analyst is now given **the actual client dataset**.

What happens?

They don't start from scratch.
Instead, they:

- Apply the same **template** of analysis they just rehearsed
- Use the same **structure** of reasoning (charts → patterns → insights)
- Follow the same **sequence of logic** they practiced earlier

*The warm-up examples have now become their **mental playbook***

That's how Auto-CoT works

So far, we've explored how to **prompt the model** to think better — using Zero-shot, Few-shot, and Auto-CoT.

But what if the model still gives *different* answers each time? Or sometimes gets it wrong, even with good reasoning?

Problem: Even CoT Isn't Always Consistent

Ask the same CoT prompt **multiple times**, and sometimes you'll get **different answers.**

Sometimes, the model's reasoning chain *looks perfectly logical*, but the **final answer is wrong**, **not because of reasoning**, but because the model doesn't have the **correct background knowledge**.

For example, if the model doesn't remember the exact income tax slabs, it may apply:

- The wrong percentage for a slab
- An outdated regime (old vs new)

This leads to errors **even with step-by-step reasoning**.

Why This Happens?

Large Language Models (LLMs) don't think deterministically.

They:

- Predict words based on probabilities
- May generate slightly different outputs **each time**, especially with CoT, which has many valid reasoning paths

So, while the reasoning is there, the answer might not be **consistent**.

> "So we taught the model *how* to think,
> But now we need to help it *think reliably.*"

That brings us to —
Self-Consistency.

Self-Consistency:

Imagine this scenario:

You're solving a math problem, and you're not 100% sure.
What do you do?
- You solve it once.
- Then maybe you solve it again in a slightly different way.
- Maybe even a third time — just to be sure.

If your **final answer is the same in each case**, you feel more confident.
If it **differs**, you go back and check which method was more trustworthy.

That is exactly how **Self-Consistency** works — but for a language model.

Self-consistency is a technique used after Chain of Thought prompting to increase the reliability of answers.
Instead of relying on **one single response**, we ask the model to:

- Generate multiple answers to the same question (using randomness),
- Observe the variation in those answers,
- Then select the **most common** final answer.

Goal: Reduce mistakes and increase accuracy by trusting what the model says **most consistently**.

Prompt:

Q: I commented — "If your salary is ₹7,00,000 per year, you don't have to pay any tax." Is this correct?

Let's think step by step.

Attempt 1:

- ₹7,00,000 is below ₹10,00,000
- So maybe no tax is needed
- **Answer: Yes, it's correct**

Looks reasonable, but **wrong logic** (10 lakh isn't the exemption limit)

Attempt 2:

- ₹7,00,000 income
- Standard deduction = ₹50,000 → Net = ₹6,50,000
- Under the old regime, some tax is there
- But under the new regime with rebate under 87A, it might be zero

Answer: Depends on the tax regime

Technically **not wrong**, but the model didn't mention **rebate limits properly**.

Attempt 3:

- ₹7,00,000 salary
- Section 87A gives a rebate up to ₹7L on taxable income
- So, final tax = 0

Answer: Yes, the comment is correct if no other income

Correct reasoning — but only **if the user opts for the right regime**.

As you can see, reasoning is attempted…
But some versions might go wrong in calculation, logic, or step interpretation.

That's because:

- The model's sampling is **non-deterministic** (especially in creative or multi-step prompts),
- So even CoT isn't always **stable**.

Self-Consistency Fixes That

With Self-Consistency, instead of asking just once, we:

1. Run the **same prompt multiple times** (say 5–10 generations),
2. Collect **all final answers**, and
3. Pick the answer that appears **most frequently**.

That "most frequent" answer is assumed to be the one where the model's internal reasoning **agrees with itself.**

So we've learned two powerful things:

1. Chain of Thought (CoT): Helps the model reason step-by-step.

2. Self-Consistency: Helps improve answer accuracy by selecting the most frequent answer from multiple tries."

But what happens when we **combine them**?

That's where **Self-Consistent CoT** comes in —
It brings together the best of both worlds:

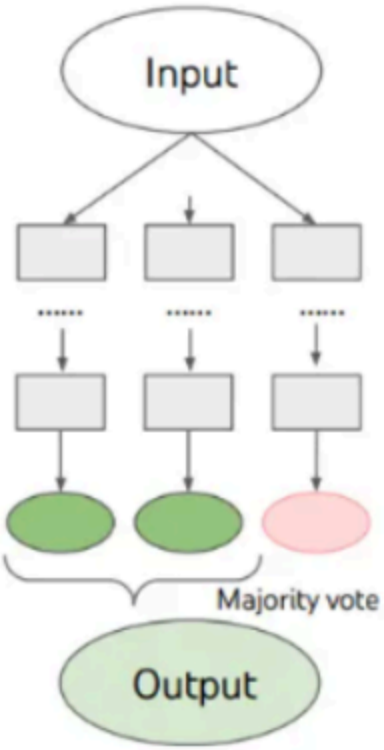
Self-Consistent CoT:

Self-Consistent Chain of Thought is a technique where we:

1. Prompt the model with a Chain of Thought style prompt
2. Ask the model to generate multiple different reasoning paths.
3. Collect all the final answers
4. Pick the one that appears most frequently

It's like:

Multiple students solving the same problem in steps. Some may make minor errors...
But if a majority reaches the same correct answer, that's the one we go with.



Self Consistent with COT

For Example:
Prompt:

Hello,

I have discovered a major security vulnerability in your system. Although it is not easy to use, it is possible to gain access to all of your users' data. I have attached a proof of concept. Please fix this issue as soon as possible.

Cheers,
Donny

Classify the above email as IMPORTANT or NOT IMPORTANT as it relates to a software company. Let's think step by step.

AI Output

The sender has discovered a major security vulnerability in the system. This is important because it could lead to a data breach. I would classify this email as IMPORTANT.

AI Output

The sender has discovered a major security vulnerability in the system, but they have not provided any proof of concept, and it is not easy to use. I would classify this email as NOT IMPORTANT.

AI Output

The sender is likely a hacker and may have access to the system. This is important because it could lead to a data breach. I would classify this email as IMPORTANT.

This is what a Self-consistent chain of thought is, in which LLM generates multiple paths of output and selects the output that is correct more.

As of now, we've seen how to make the model think step-by-step,
and how to improve the reliability of its thinking.
But here's a big question:
What if there are **multiple valid ways** to reach a solution — not just one?"

Let's say the model needs to:

- Solve a riddle
- Write a story
- Make a decision
- Plan an itinerary

Sometimes, **one reasoning path isn't enough.**

Limitation of Self-Consistency

Self-Consistency works great when the **problem has one correct answer**,
like:

- What is 17 × 12?
- Who is the oldest in the family?
- How many apples will be left?

These are **factual** or **logical** problems with one clear path.

But what if the problem isn't so clear?

What if it's a puzzle, a design, a strategy task — where:

- There's **no single right answer**
- There are **many possible ways** to approach the problem
- The quality of the answer depends on **comparing different ideas**

Then Self-Consistency doesn't help much.

For instance:
Self-Consistency is like asking the model to flip a coin 10 times and trust the side that comes up most.

> But what if we want the model to **think like a strategist**?
> explore multiple options, weigh them, and then decide?"

That's where we need a **more advanced and creative thinking technique,** i.e, Tree of Thought

Tree of Thought:

"Train the model not just to answer, but to think like a planner."

Imagine you're part of a team in a startup incubator, and your task is to **pitch a startup idea** to investors.

You don't just go with the first idea that comes to mind.

Instead, you think like this:

Step 1: Generate Multiple Ideas

- Idea 1: A grocery delivery app
- Idea 2: A mental wellness platform
- Idea 3: A smart water bottle that tracks hydration

Each idea is like the **first branch** of your tree.

Step 2: Explore Each Idea Further

Now for each idea, you ask:

- What's the target market?
- Is the business model sustainable?
- Is the tech feasible?
- What's the unique edge?

Each answer becomes a **sub-branch**.

Step 3: Evaluate Each Path

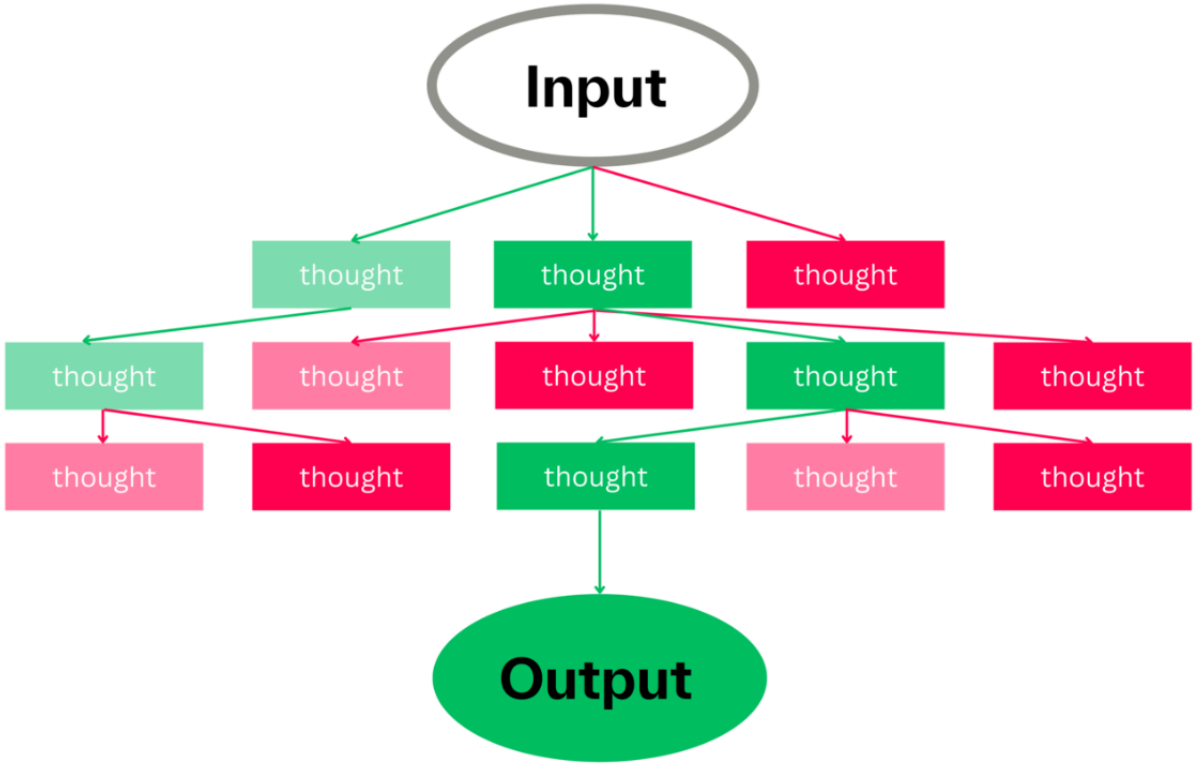
- The grocery delivery app: Market is crowded. Weak differentiation.
- The wellness platform: High demand, needs strong content pipeline. Maybe.
- The smart bottle: Unique niche + health trend + IoT potential.

You pick the **best-developed path** and finalize that as your pitch.

That's Tree of Thought Thinking

You **don't commit early**.
You **expand multiple ideas**.
You **grow each one** with logic.
Then you **evaluate** and **choose** the most promising path.

It's not just linear step-by-step — it's structured **exploration and comparison**.



Tree of Thoughts-Prompting
(ToT)

Why Tree of Thought Exists

"From calculators to planners — evolving the way models think."

Let's Revisit What We Had So Far:

- **Chain of Thought (CoT)** helped us teach the model to "think out loud," like a human solving a problem on paper:
    - One thought → leads to the next → and so on…
    - Great for arithmetic, logic, and factual QA
- **Self-Consistent** made that reasoning more **trustworthy**:
    - Run the model multiple times
    - Compare outcomes
    - Pick the answer that comes up most often

Together, they gave us a system where the model thinks *clearly* and *accurately*.

But here's the **catch**...

The Limitation of Linearity

Both CoT and Self-Consistency operate on a **linear track**:

- One idea → One reasoning path → One conclusion
- Or, generate **many** linear paths and pick the one that repeats

But **real-world problems** — especially creative, open-ended, or strategic ones — **don't follow a straight line**.

Let's take examples:

- Writing a story ending
- Planning an event with constraints
- Solving a logic puzzle with multiple candidate moves
- Making a business decision with competing priorities

In such cases, you don't just follow **one thread** — you brainstorm, branch out, compare alternatives, and pick the best one.

This is where both CoT and Self-Consistency **hit a wall**:

- They can't **compare multiple incomplete ideas**
- They don't **pause midway** to evaluate or prune a bad path
- They can't **adapt** based on what they learn during exploration

Then the Tree of Thought was created to solve these **higher-order thinking problems**, where:

- There are **multiple solution paths**
- Reasoning involves **exploring, rejecting, or refining** ideas

- The model must **branch**, **backtrack**, or **merge thoughts** — not just walk a straight line

For Example:
Prompt:

"Generate 3 plot branches for a mystery novel where the detective is the murderer."

ToT Exploration:

Branch 1:
The detective plants evidence to frame suspects.
Climax: A rookie cop discovers their handwriting on a forged note.

Branch 2:
The detective kills to "solve" cold cases and gain fame.
Twist: Their spouse uncovers a hidden victim list.

Branch 3:
The detective has dissociative identity disorder.
Reveal: Security footage shows them at the crime scene in a different persona.

ToT Selection: Branch 3 (most original).

Comparison Table: COT vs Self Consistent vs TOT

Let's Understand Through a Comparison

- Use **CoT** when the model just needs to think clearly,
- **Self-consistency** when you want it to be more reliable,
- and **Tree of Thought** when you want to explore, compare, and reason through **multiple possibilities**.

Conclusion: Key Takeaways from Advanced Prompt Engineering Techniques

1. **Basic prompting is not enough for complex reasoning tasks.**
   Techniques like Zero-Shot, One-Shot, Few-Shot, and Role-Based prompting give foundational control, but they lack depth for analysis, planning, and justification.

2. **Chain of Thought (CoT) helps models simulate human reasoning.**
   It enables step-by-step thinking, improving transparency and accuracy, especially for logic-based or multi-step problems.

3. **Self-Consistency enhances reliability by reducing randomness.**
   By sampling multiple reasoning paths and selecting the most frequent outcome, it boosts

confidence in the model's answers.

4. **Tree of Thought (ToT) introduces structured exploration.**
   It allows the model to branch out into multiple ideas, evaluate alternatives mid-process, and choose the best path—ideal for open-ended, creative, or strategic tasks.

5. **Different techniques suit different problem types.**
   CoT is best for logic-heavy problems, Self-Consistency for accuracy in unstable outputs, and ToT for tasks that need exploration, comparison, and planning.